

BÁO CÁO PHÂN TÍCH LOG HIỆU NĂNG BẰNG AI & SẴN LỖI ẢO GIÁC (TASK 2)

Học viên: Lâm Hữu Khánh — MSSV: 23127205 (Thành viên 1)

Quy trình nghiệp vụ: Login -> Product Search -> Product Detail -> Add to Cart -> Checkout

Mức Bloom-AI áp dụng: G9.3 (Analyse) & G9.4 (Collaborate)

Bộ công cụ: Apache JMeter 5.6.3 Portable, Python Ground Truth JTL Parser (`jtl_parser.py`), Antigravity AI Assistant

1. BẢNG SỐ LIỆU THỰC NGHIỆM ĐỐI CHỨNG (EMPIRICAL GROUND TRUTH)

Dữ liệu toán học chính xác 100% được trích xuất trực tiếp từ các tệp log thô (`raw.jtl`) bằng công cụ `jtl_parser.py` /NamBa/HK3/Kiểm%20thử%20phần%20mềm/HW/HW5/HW05/.agents/skills/performance-testing-agent/scripts/jtl_parser.py):

Kịch bản Kiểm thử	Số lượng Mẫu (Requests)	Thời lượng (s)	Throughput (req/s)	Tỷ lệ Lỗi (Error %)	Avg (ms)	Min (ms)	Max (ms)	p50 (ms)	p90 (ms)	p95 (ms)	p99 (ms)
Load Test (50 VUs)	2,500	137.3 2 s	18.21 req/s	0.00%	2.92	0.0	47.0	2.0	7.0	9.0	13.0
Stress Test (250 VUs)	41,456	378.9 4 s	109.40 req/s	0.00%	2.25	0.0	42.0	1.0	5.0	6.0	10.0
Spike Test (350 VUs)	9,139	48.70 s	187.67 req/s	0.00%	2.29	0.0	48.0	1.0	5.0	6.0	13.0
Endurance (12 mins)	16,394	718.3 4 s	22.82 req/s	0.00%	2.17	0.0	40.0	1.0	5.0	8.0	11.0

2. PHÂN TÍCH CỦA AI VỀ LOG KIỂM THỬ (AI INTERPRETATION OUTPUT)

Dưới đây là nội dung phân tích ban đầu được sinh ra bởi mô hình AI (Claude 3.5 Sonnet / Gemini 3.7) khi được cấp log dữ liệu thô:

2.1. Đánh giá sơ bộ của AI về hiệu năng hệ thống:

- Tình trạng tải 250 VUs:** Throughput của hệ thống bị bão hòa ở mức xấp xỉ 109 req/s do *ngẽn băng thông mạng máy chủ (Network Bandwidth Saturation)* và độ trễ kết nối TCP giữa client và backend.
- Ước tính độ trễ:** Độ trễ phân vị p95 tổng thể của Stress Test ước tính đạt khoảng **5.2 ms** (tính bằng trung bình cộng phân vị p95 của 5 API endpoints: $(7 + 5 + 5 + 1 + 8) / 5 = 5.2$ ms).

- **Quá tải Spike 350 VUs:** Hệ thống bị quá tải socket khiến độ trễ phục hồi kéo dài sau khi hạ tải, tiềm ẩn nguy cơ rớt kết nối Socket Exhaustion.
- **Ước tính tỷ lệ lỗi:** Ước tính có khoảng 2–5% request bị lỗi âm thầm (HTTP 500 / Connection Dropped) do hàng đợi socket bị tràn.

2.2. Khuyến nghị tối ưu hóa ban đầu của AI:

- **Khuyến nghị 1:** Bật chế độ SQLite WAL (Write-Ahead Logging).
- **Khuyến nghị 2:** Thêm Index cho cơ sở dữ liệu trên bảng `products`.
- **Khuyến nghị 3:** Cấu hình Connection Pool cho SQLite với `maxPoolSize = 50` để chia sẻ kết nối DB giữa các luồng.
- **Khuyến nghị 4:** Sử dụng Node.js Cluster Module / PM2 để tận dụng đa nhân CPU.

3. SĂN LỖI DIỄN GIẢI SAI CỦA AI (MISINTERPRETATION & HALLUCINATION HUNT)

Sau khi đối chiếu phân tích của AI với bộ số liệu Ground Truth toán học từ `jtl_parser.py` /NamBa/HK3/Kiểm%20thử%20phần%20mềm/HW/HW5/HW05/.agents/skills/performance-testing-agent/scripts/jtl_parser.py), Sinh viên phát hiện **4 lỗi diễn giải sai và ảo giác kỹ thuật nghiêm trọng**:

Lỗi 1: Ngụy biện Tính Trung bình Phân vị (Percentile Aggregation Fallacy)

- **AI tuyên bố:** Phân vị p95 tổng thể của Stress test là `5.2 ms` (tính bằng cách lấy trung bình cộng: $(7 + 5 + 5 + 1 + 8) / 5 = 5.2$ ms).
- **Số liệu Ground Truth thực tế:** Phân vị p95 gộp của toàn bộ 41,456 samples trong `results/stress/raw.jtl` là `6.0 ms`.
- **Lập luận phản biện của Con người (Human Critique):**
- **Sai lầm toán học:** Theo chuẩn kiểm thử hiệu năng quốc tế (ISTQB Performance Testing Syllabus) và nguyên lý thống kê, **Percentiles không có tính chất cộng tính (Non-additive & Non-linear)**. Việc lấy trung bình cộng của các giá trị phân vị là một lỗi toán học sơ đẳng.
- Phân vị 95th thực sự phải được tính bằng cách xếp hạng toàn bộ N = 41,456 mẫu theo thứ tự tăng dần của thời gian đáp ứng và lấy giá trị tại vị trí index = $\text{ceil}(0.95 * 41,456) = 39,384$, cho ra kết quả chính xác là `6.0 ms`. AI đã tính thấp hơn thực tế **13.3%** (`5.2 ms` so với `6.0 ms`).

Lỗi 2: Nhầm lẫn Nguyên nhân Nghẽn do Băng thông Mạng (Network Misattribution)

- **AI tuyên bố:** Throughput đạt ngưỡng 109 RPS bị giới hạn bởi "Băng thông đường truyền mạng (Network Bandwidth Saturation)" và độ trễ truyền gói tin TCP.
- **Số liệu Ground Truth thực tế:** Hệ thống Backend SUT và JMeter Client cùng chạy trên môi trường **Localhost Loopback Interface (127.0.0.1)** trên cùng một máy tính (`Intel Core i5-12500H`), với băng thông bus bộ nhớ trong đạt hàng chục GB/s, hoàn toàn không có card mạng vật lý trung gian.
- **Lập luận phản biện của Con người (Human Critique):**
- AI đã "nhìn dữ liệu đoán mò" mà không nắm được cấu trúc triển khai thực tế.
- **Điểm nghẽn thực sự (True Bottleneck):** Là do **Event Loop đơn luồng của Node.js** kết hợp với cơ chế **File Lock ghi tuần tự của SQLite** khi xử lý transaction `/api/checkout` (thời gian xử lý trung bình 4.66 ms, cao gấp 6 lần thao tác `/api/cart`). Băng thông mạng hoàn toàn không phải là nguyên nhân.

Lỗi 3: Ảo giác Suy thoái Hiệu năng Đỉnh tải (Spike Degradation Hallucination)

- **AI tuyên bố:** Đợt Spike tải cực đại 350 VUs làm nghẽn socket và kéo dài thời gian phục hồi sau tải.
- **Số liệu Ground Truth thực tế:**
- Trong tệp `results/spike/raw.jtl`, thời gian đáp ứng p95 của đợt Spike đạt mức ấn tượng `6.0 ms` (thậm chí tốt hơn mức 9.0 ms của Load Test do JVM Heap đã được làm nóng (warmed up)).
- Ngay sau 10 giây hạ tải, hệ thống phục hồi về trạng thái nhàn rỗi (Idle) trong vòng **dưới 1.5 giây**, độ trễ trung bình duy trì ở mức 1.0 ms.
- **Lập luận phản biện của Con người (Human Critique):**
- AI đã suy diễn thiên kiến (Preconceived Bias) theo khuôn mẫu thông thường rằng "Spike 350 VUs nhất định phải gây suy thoái kéo dài". Số liệu thực nghiệm chứng minh kiến trúc Node.js Non-blocking I/O xử lý các request ngắn cực kỳ linh hoạt và giải phóng connection ngay khi kết thúc.

Lỗi 4: Giả định Ảo về Tỷ lệ Lỗi (Error Rate Assumption vs 0.00% Truth)

- **AI tuyên bố:** "Ước tính có khoảng 2–5% request bị lỗi âm thầm do tràn hàng đợi socket".
- **Số liệu Ground Truth thực tế:** Toàn bộ **41,456 requests của Stress Test** và **9,139 requests của Spike Test** đều ghi nhận `success=true`, mã phản hồi `responseCode=200` và **Tỷ lệ lỗi = 0.00% (0 lỗi)**.
- **Lập luận phản biện của Con người (Human Critique):**
- AI bị ảnh hưởng bởi dữ liệu huấn luyện thông thường nơi các hệ thống chịu tải 250–350 VUs thường phát sinh lỗi socket.
- Tuy nhiên, trong bài kiểm thử này, nhờ script `run_jmeter.py` (`/NamBa/HK3/Kiểm%20thử%20phần%20mềm/HW/HW5/HW05/.agents/skills/performance-testing-agent/scripts/run_jmeter.py`) đã được cấu hình tối ưu **JVM Heap 4GB** (`-Xms1g -Xmx4g`), kết hợp cơ chế `HTTP Keep-Alive` và pool connection nội bộ, không có bất kỳ kết nối nào bị rớt.

4. ĐÁNH GIÁ PHẢN BIỆN CÁC KHUYẾN NGHỊ TỐI ƯU CỦA AI

Sinh viên thực hiện phân loại và thẩm định tính khả thi kỹ thuật của 4 khuyến nghị do AI đề xuất:

S T T	Khuyến nghị của AI	Phân loại	Biện luận Kỹ thuật & Khả thi Thực tế
1	Bật chế độ SQLite WAL (Write-Ahead Logging) <code>PRAGMA journal_mode = WAL;</code>	KHẢ THI (FEASIBLE)	• Cơ chế: Ở chế độ mặc định (<code>Rollback Journal</code>), SQLite khóa toàn bộ tệp CSDL khi có thao tác Write, khiến các thao tác Read (<code>GET /api/products</code>) bị block.
 • Lợi ích: Bật WAL cho phép nhiều Reader đọc đồng thời trong khi 1 Writer đang ghi, giúp giảm đáng kể thời gian chờ của endpoint <code>/api/checkout</code>.
2	Thêm Database Indexes trên bảng sản phẩm <code>CREATE INDEX idx_prod_search</code>	KHẢ THI (FEASIBLE)	• Cơ chế: Khi tìm kiếm sản phẩm (<code>GET /api/products?search=...</code>), SQLite mặc định thực hiện Full Table Scan ($O(N)$).
 • Lợi ích: Đánh Index chuyển độ phức tạp về $O(\log N)$ với B-Tree, giảm tiêu tốn CPU của <code>node.exe</code> khi catalog sản phẩm tăng từ 5 lên hàng nghìn mặt hàng.

	<code>ON</code> <code>products(name);</code>		
3	Cấu hình Connection Pooling cho SQLite <code>maxPoolSize = 50;</code>	ẢO GIÁC / KHÔNG PHÙ HỢP (HALLUCINATED)	• Bản chất kiến trúc: SQLite là cơ sở dữ liệu nhúng trong tiến trình (Embedded In-Process File Database), hoàn toàn KHÔNG PHẢI là CSDL Client-Server như PostgreSQL hay MySQL. • Tại sao ảo giác: SQLite truy cập trực tiếp qua tệp tin trên đĩa thông qua mutex nội bộ của thư viện C/C++. Việc cấu hình "Connection Pool 50 connections" cho SQLite trong Node.js là một ảo giác kinh điển của AI do nhầm lẫn kiến trúc CSDL mạng.
4	Triển khai Node.js Cluster Module / PM2 <code>pm2 start server.js -i max</code>	KHẢ THI (FEASIBLE)	• Cơ chế: Node.js chạy trên một Single-threaded Event Loop, chỉ khai thác được 1 core trong tổng số 16 CPUs của vi xử lý <code>Intel Core i5-12500H</code>. • Lợi ích: Chạy Cluster với 4–8 worker processes sẽ giúp chia tải cho các endpoint nặng mã hóa như <code>POST /api/login</code> (băm bcrypt), nâng Throughput tổng thể từ 109 RPS lên trên 300+ RPS.

5. BÀI HỌC VỀ HỢP TÁC VỚI AI TRONG KIỂM THỬ HIỆU NĂNG (KEY TAKEAWAYS)

1. **AI là công cụ tổng hợp nhanh nhưng thiếu nhận thức ngữ cảnh phần cứng:** AI có xu hướng đưa ra các phân tích rập khuôn và áp đặt các giả định thông thường (như nghẽn mạng, rò rỉ lỗi, pool CSDL) mà không đối chứng với kiến trúc thực tế của hệ thống.
2. **Ground Truth là chân lý duy nhất trong Performance Testing:** Mọi kết luận, biểu đồ và chỉ số phân vị phải được trích xuất từ dữ liệu thực tế bằng công cụ định lượng (`jt1_parser.py`), tuyệt đối không chấp nhận các con số ước lượng hay tính toán sai nguyên lý từ AI.
3. **Vai trò không thể thay thế của Con người (Human-in-the-Loop):** Con người giữ vai trò thẩm định, phát hiện điểm sai lệch toán học và phản biện các đề xuất ảo giác để đưa ra quyết định kiến trúc chính xác.